55/55 Great!

Homework 6

William Flanders*
Haverford College
(Dated: March 22, 2023)

+25

1. NEWMAN EXERCISE 8.4 A

We look to model a pendulum system when we can't use the small angle approximation $sin\theta \approx \theta$ i.e. a non-linear pendulum. Beginning with the equation for a pendulum,

$$ml\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -mg\sin\theta \tag{1}$$

where $g = 9.81\frac{m}{s^2}$, and l and m is the length and mass of the pendulum respectively. We note that it is a second order, non-linear, ordinary differential equation. We can cancel out the mass, and solve for the differential term.

$$\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -\frac{g}{l}\sin\theta. \tag{2}$$

Looking to solve this numerically and create a plot of t vs. θ, we will employ Runge-Kutta 4 to solve the ODE. To this end, we will want to turn the second order ODE into two coupled first order ODE's. We can accomplish this by setting

$$\omega = \frac{\mathrm{d}\theta}{\mathrm{d}t} \tag{3}$$

where now our equation for the pendulum becomes the system of equations

$$\frac{\mathrm{d}\theta}{\mathrm{d}t} = \omega \tag{4}$$

$$\frac{\mathrm{d}\omega}{\mathrm{d}t} = -\frac{g}{l}\sin\theta. \tag{5}$$

We are now ready to create a python program to model the system. We will look to model a system where the length is 10cm and our time interval is from $t = 0$ s to $t = 10$ s. Our initial conditions are $\theta = 179°$ and $t = 0$ s. We start with pseudo code,

```
import dependencies

initialize constants

# define the function that we wish to solve
def f(r,t):
    # since we are solving a system of coupled
    # ode's, we will take in a vector r that
    # contains both functions of theta and omega

define interval
define stepsize
calculate height

initial conditions

Runge-Kutta 4 algorithm
    return values of theta

plot time vs. theta
```

Our outputted plot is shown in Figure 1.

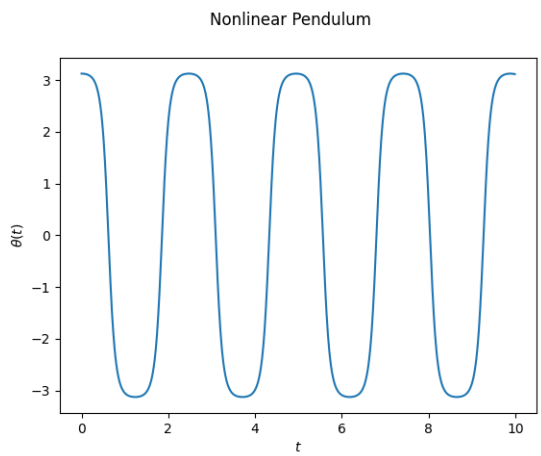


FIG. 1: Plot of θ vs. time. θ is measured in radians and t in seconds.

The pendulum's motion is clearly still sinusoidal, however, it is obviously nonlinear. The body of the waves are much longer vertically and stretched out compared to the peaks and troughs which seemed to have been squished. Physically, the pendulum will appear to move faster when it is in motion and will hang in the air for longer when it has reached the top.

2. NEWMAN EXERCISE 8.5 A,B

We look to model a nonlinear, undamped, driven pendulum system. This system takes the equation

*Electronic address: wflanders@haverford.edu; URL:

$$\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -\frac{g}{l}\sin\theta + C\cos\theta\sin\Omega t \tag{6}$$

where C and Ω are constants, $g = 9.81\frac{m}{s^2}$, and l is the length of the pendulum. Just as in the previous example, we can turn the second order ODE into a system of coupled first order ODE's, employ Runge-Kutta 4, and output figures to qualitatively observe the behavior. Our coupled system can be modeled by

$$\frac{\mathrm{d}\theta}{\mathrm{d}t} = \omega \tag{7}$$

$$\frac{\mathrm{d}\omega}{\mathrm{d}t} = -\frac{g}{l}\sin\theta + C\cos\theta\sin\Omega t. \tag{8}$$

We will use code that mimics the previous exercise's with small tweaks. We will be looking to model the system from $t = 0$s to $t = 100$s. Our initial conditions will be $\theta = 0°$ and $t = 0$s.

We start with pseudo code

```
import dependencies

initialize constants

# define the function that we wish to solve
def f(r,t):
    # our function will now include
    # the driving force in addition
    # to the equation for the pendulum

define interval
define stepsize
calculate height

initial conditions

Runge-Kutta 4 algorithm
    return values of theta

plot time vs. theta
```

We will first consider when $C = 2s^{-2}$ and $\Omega = 5s^{-1}$. Our outputted plot is shown in Figure 2.

Qualitatively, we see that the max amplitude the pendulum reaches is around 0.04 radians $\approx 2.29°$. Physically, the pendulum is barely moving which indicates that the system is strongly out of resonance. The frequency at which the driving force oscillates differs fairly from the natural frequency of the pendulum.

Lets look for when the system is at resonance. As previously stated, this will occur when the driving frequency is equivalent to the natural frequency,

$$\omega_0 = \omega_{drive} \tag{9}$$

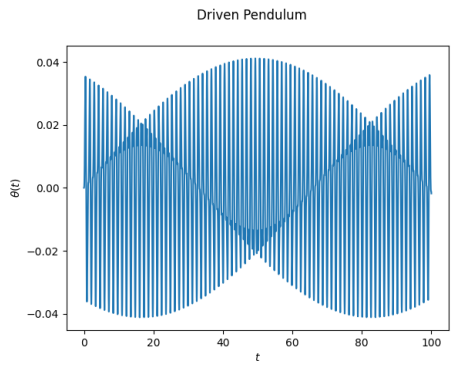


FIG. 2: Driven pendulum when $C = 2s^{-2}$ and $\Omega = 5s^{-1}$. θ is measured in radians and t in seconds.

$$\omega_{drive} = \sqrt{\frac{g}{l}} \tag{10}$$

which is roughly equivalent to $9.9045s^{-1}$. Setting our constant Ω equivalent to this in our program, we output the following plot as shown in Figure 3.

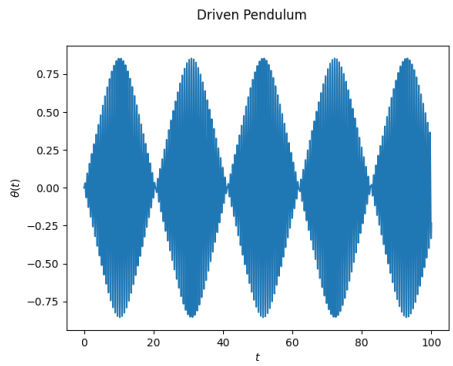


FIG. 3: Driven pendulum when $C = 2s^{-2}$ and $\Omega = 9.9045s^{-1}$. θ is measured in radians and t in seconds.

Qualitatively, the plot shows the emergence of beats. This usually occurs when we approach resonance without actually fully achieving it. Furthermore, we can conclude that we are close since the amplitude rises to roughly 0.8 radians $\approx 45.84°$. At resonance, we would expect oscillations that continue to grow in amplitude as time increases since energy is being passed into the system perfectly from the driving force. This isn't the case from the plot. Its possible that computer rounding error prevents the system from ever being perfectly at its resonant frequency. Even when Ω is calculated within the program using Equation 10 instead of hard coding in the value $9.9045s^{-1}$, we still output beats instead of the expected plot.

This is what the plot is supposed to look like and I don't think there is an error in your code/computer error. I think that the beating is expected given the driving force itself has that envelope shape. This is unlike a driving force that is just a single sin wave.

3. NEWMAN EXERCISE 8.2

+25

We want to investigate predator-prey systems using the Lotka-Volterra equations. The equations are a system of coupled first order linear ordinary differential equations.

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \alpha x - \beta xy \tag{11}$$

$$\frac{\mathrm{d}y}{\mathrm{d}t} = \gamma xy - \delta y \tag{12}$$

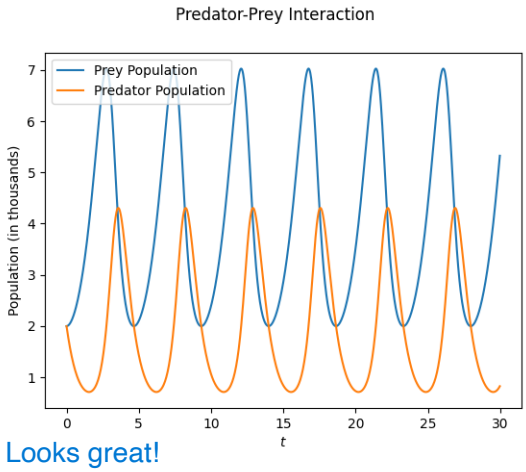
where x is the prey population, y is the predator population, and α, β, γ, and δ are parameters. From analyzing the equations, we can see how prey population increases proportional to their size, but also decreases proportional to how large the predator size is. Similarly, for the predator population, we can see that the population increases when the prey population increases, but decreases proportional to its own size. All units are measured in thousands.

We will look to model a situation where both populations start at 2,000, i.e. $x = 2$ and $y = 2$ and our time interval will be from $t = 0$ to $t = 30$. We will employ Runge-Kutta 4 to solve the coupled system. We start with pseudo code

```
import dependencies

initialize constants

# function for our coupled system
def f(r,t):
    # functions for x and y
    # contained within the
    # r vector

define interval
define stepsize
calculate height

initial conditions for x and y

Runge-Kutta 4 algorithm
    return values of x and y

plot time vs. x
plot time vs. y
```

our returned plot is shown in Figure 4.

As expected, we see that prey populations increase exponentially when predator populations are low. However, the abundance of prey now allows the predator population to skyrocket. This leads to over hunting, and the prey population starts to exponentially decrease. The loss of prey causes the predator population to fall, and the oscillatory behavior continues. With initial conditions of 2000 for both prey and predator, it appears the system is able to remain stable. Varying the initial conditions can lead to very different solutions. For example, 1000 for predators and 50 for prey would likely result in a system where the prey dies out (population approaches 0) and the predators then follow suit.



Looks great!

FIG. 4: Predator-Prey simulation with initial conditions of 2000 for both populations.

4. SURVEY

+5

The problem set took roughly 4-5 hours. The most difficult problem was Newman Exercise 8.5. I ensured that the step size was large enough such that the behavior for the plot converged, but no matter how I tried to vary the resonant frequency, I could never get resonance behavior. I theorized this was due to computer rounding and imprecision but there could also be issues with my code as well.

Problem 8.4/5

Flanders +25

Computational Physics/Astrophysics, Winter 2023: Grading Rubrics [1]

Haverford College, Prof. Daniel Grin

For coding assignments, roughly 25 points will be available per problem.

+3 1. Does the program complete without crashing in a reasonable time frame? If yes, up to +3 points.

+1 2. Does the program use the exact program files given (if given), and produce an answer in the specified format? If yes, +1 points

+2 3. Does the code follow the problem specifications (i.e numerical method; output requested etc.) Up to +2 points

+4 4. Is the answer correct? Up to+4points

+2 5. Is the code readable? Up to+2points

. 5.1. Are variables named reasonably?

. 5.2. Are the user-functions and imports used?

. 5.3. Are units explained (if necessary)?

. 5.4. Are algorithms found on the internet/book/etc. properly attributed?

[1] Inspired by rubric of D. Narayanan, U. Florida, and C. Cooksey, U. Hawaii

+3 6. Is the code well documented? +3points

. 6.1. Is the code author named?

. 6.2. Are the functions described and ambiguous variables defined?

. 6.3. Is the code functionality (i.e. can I run it easily enough?) documented?

+10 7. LaTeX writeup (up to 10 points)

- Are key figures and numbers from the problem given? (3 points)
- Is a brief explanation of physical context given? (2 points)
- If relevant, are helpful analytic scalings or known solutions given? (1 point)
- Are 3-4 key equations listed (preferably the ones solved in the programming assignment) and algorithms named? (2 points)
- Are collaborators clearly acknowledged? (1 point)
- Are any outside references appropriately cited? (1 point)

Note, even if (1), (2), (3), or (4) are not correct, one can still obtain many points via (5), (6), and (7).

Problem 8.2

Flanders +25 Great work!

Computational Physics/Astrophysics, Winter 2023:

Grading Rubrics [1]

Haverford College, Prof. Daniel Grin

For coding assignments, roughly 25 points will be available per problem.

+3 1. Does the program complete without crashing in a reasonable time frame? If yes, up to +3 points.

+1 2. Does the program use the exact program files given (if given), and produce an answer in the specified format? If yes, +1 points

+2 3. Does the code follow the problem specifications (i.e numerical method; output requested etc.) Up to +2 points

+4 4. Is the answer correct? Up to+4points

+2 5. Is the code readable? Up to+2points

. 5.1. Are variables named reasonably?

. 5.2. Are the user-functions and imports used?

. 5.3. Are units explained (if necessary)?

. 5.4. Are algorithms found on the internet/book/etc. properly attributed?

[1] Inspired by rubric of D. Narayanan, U. Florida, and C. Cooksey, U. Hawaii

+3 6. Is the code well documented? +3points

. 6.1. Is the code author named?

. 6.2. Are the functions described and ambiguous variables defined?

. 6.3. Is the code functionality (i.e. can I run it easily enough?) documented?

+10 7. LaTeX writeup (up to 10 points)

- Are key figures and numbers from the problem given? (3 points)
- Is a brief explanation of physical context given? (2 points)
- If relevant, are helpful analytic scalings or known solutions given? (1 point)
- Are 3-4 key equations listed (preferably the ones solved in the programming assignment) and algorithms named? (2 points)
- Are collaborators clearly acknowledged? (1 point)
- Are any outside references appropriately cited? (1 point)

Note, even if (1), (2), (3), or (4) are not correct, one can still obtain many points via (5), (6), and (7).